Student: Muhammed Akif Sevilgen **Student number:** 26420101064

CSE513 Assignment 1 — CPU Scheduling Simulator

Working website: https://makifsevilgen.github.io/CSE513-Scheduler/

Source code repository: https://github.com/makifsevilgen/CSE513-Scheduler

1. Overview

The simulator is a single self-contained HTML page (HTML, CSS and JavaScript, no backend) that implements FCFS, non-preemptive SJF, preemptive SRTF and Round Robin with a configurable quantum. It follows the assignment model: one CPU, one CPU burst per process, no I/O, zero-cost context switches, integer times. The rules are applied exactly as specified: ties are broken by earlier arrival and then by earlier position in the input list; SRTF preempts only for a strictly shorter remaining time; in Round Robin, processes arriving at a quantum boundary join the queue before the preempted process, and finished processes never rejoin. Idle intervals are shown when the ready queue is empty. Intervals are [start, end). Invalid input (duplicate or empty IDs, negative or non-integer arrival, burst or quantum below 1) is rejected with an explanation.

Metrics: turnaround = completion − arrival; waiting = turnaround − burst; response = first start − arrival. In the Gantt charts below, each box is one uninterrupted run; in Round Robin each box is one quantum slice, so consecutive slices of the same process remain visible as separate boxes.

Method. For every example the timeline and metrics were first worked out by hand (the “Hand” values), then compared with the simulator output (the “Sim” values). A cell shows *hand / sim*. An automatic script produced the simulator values and the match flags.

2. Worked examples (Workloads A and B)

Workload A: P1 (arrival 0, burst 8), P2 (1, 4), P3 (2, 2), P4 (4, 1). Workload B: P1 (2, 3), P2 (2, 1), P3 (8, 2), P4 (9, 1). Round Robin uses quantum 2 in the examples.

Example 1: Workload A, FCFS

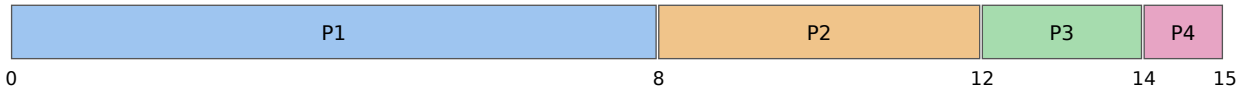
Input

Process	Arrival	CPU burst
P1	0	8
P2	1	4
P3	2	2
P4	4	1

Hand-calculated timeline: P1 0–8, P2 8–12, P3 12–14, P4 14–15.

FCFS picks the earliest-arriving ready process and never preempts. P1 (arrival 0) runs to completion at 8. P2, P3 and P4 are all waiting; by arrival order they run P2 (8–12), P3 (12–14), P4 (14–15). Start times equal waiting times because each process runs once without interruption.

Simulator output (Gantt chart) and **metrics** (hand / sim):



ID	Completion	Turnaround	Waiting	Response
P1	8 / 8	8 / 8	0 / 0	0 / 0
P2	12 / 12	11 / 11	7 / 7	7 / 7
P3	14 / 14	12 / 12	10 / 10	10 / 10
P4	15 / 15	11 / 11	10 / 10	10 / 10
Average	–	10.50 / 10.50	6.75 / 6.75	6.75 / 6.75

Comparison: the hand-calculated timeline and every metric match the simulator output exactly; no discrepancy.

Example 2: Workload B, FCFS

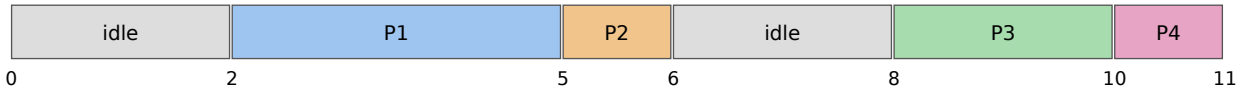
Input

Process	Arrival	CPU burst
P1	2	3
P2	2	1
P3	8	2
P4	9	1

Hand-calculated timeline: idle 0–2, P1 2–5, P2 5–6, idle 6–8, P3 8–10, P4 10–11.

The ready queue is empty until t=2, so the CPU is idle 0–2. P1 and P2 arrive together at 2 (a tie); equal arrival times are broken by input position, so P1 runs first (2–5), then P2 (5–6). The CPU is idle again 6–8. P3 arrives at 8 and runs 8–10; P4 (arrived 9) runs 10–11.

Simulator output (Gantt chart) and **metrics** (hand / sim):



ID	Completion	Turnaround	Waiting	Response
P1	5 / 5	3 / 3	0 / 0	0 / 0
P2	6 / 6	4 / 4	3 / 3	3 / 3
P3	10 / 10	2 / 2	0 / 0	0 / 0
P4	11 / 11	2 / 2	1 / 1	1 / 1
Average	–	2.75 / 2.75	1.00 / 1.00	1.00 / 1.00

Comparison: the hand-calculated timeline and every metric match the simulator output exactly; no discrepancy.

Example 3: Workload A, SJF (non-preemptive)

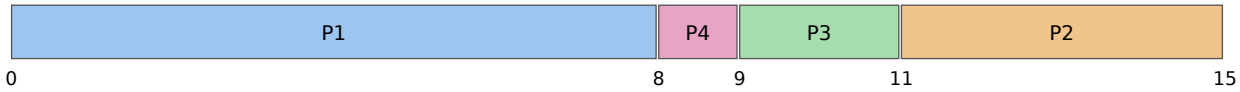
Input

Process	Arrival	CPU burst
P1	0	8
P2	1	4
P3	2	2
P4	4	1

Hand-calculated timeline: P1 0–8, P4 8–9, P3 9–11, P2 11–15.

At t=0 only P1 is ready, so it starts and cannot be preempted; it finishes at 8. At t=8 the ready processes are P2 (burst 4), P3 (2) and P4 (1). The shortest burst is chosen each time the CPU becomes free: P4 (8–9), P3 (9–11), P2 (11–15).

Simulator output (Gantt chart) and **metrics** (hand / sim):



ID	Completion	Turnaround	Waiting	Response
P1	8 / 8	8 / 8	0 / 0	0 / 0
P2	15 / 15	14 / 14	10 / 10	10 / 10
P3	11 / 11	9 / 9	7 / 7	7 / 7
P4	9 / 9	5 / 5	4 / 4	4 / 4
Average	–	9.00 / 9.00	5.25 / 5.25	5.25 / 5.25

Comparison: the hand-calculated timeline and every metric match the simulator output exactly; no discrepancy.

Example 4: Workload B, SJF (non-preemptive)

Input

Process	Arrival	CPU burst
P1	2	3
P2	2	1
P3	8	2
P4	9	1

Hand-calculated timeline: idle 0–2, P2 2–3, P1 3–6, idle 6–8, P3 8–10, P4 10–11.

Idle 0–2. At t=2 P1 (burst 3) and P2 (burst 1) are both ready: the shorter job P2 runs first (2–3), then P1 (3–6). Idle 6–8. P3 arrives at 8 and runs to 10 (P4 arriving at 9 cannot preempt in a non-preemptive policy); then P4 runs 10–11.

Simulator output (Gantt chart) and **metrics** (hand / sim):



ID	Completion	Turnaround	Waiting	Response
P1	6 / 6	4 / 4	1 / 1	1 / 1
P2	3 / 3	1 / 1	0 / 0	0 / 0
P3	10 / 10	2 / 2	0 / 0	0 / 0
P4	11 / 11	2 / 2	1 / 1	1 / 1
Average	–	2.25 / 2.25	0.50 / 0.50	0.50 / 0.50

Comparison: the hand-calculated timeline and every metric match the simulator output exactly; no discrepancy.

Example 5: Workload A, SRTF (preemptive)

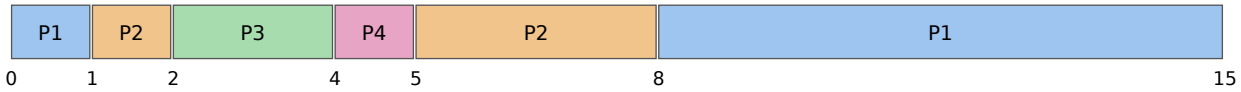
Input

Process	Arrival	CPU burst
P1	0	8
P2	1	4
P3	2	2
P4	4	1

Hand-calculated timeline: P1 0–1, P2 1–2, P3 2–4, P4 4–5, P2 5–8, P1 8–15.

t=1: P2 arrives with 4 < P1's remaining 7, so P2 preempts. t=2: P2 has 3 left and P3 arrives with 2 (strictly shorter), so P3 preempts and runs 2–4. t=4: P3 is done; P4 (1) is shorter than P2 (3) and P1 (7), so P4 runs 4–5. Then P2 finishes 5–8 and P1 finishes 8–15.

Simulator output (Gantt chart) and **metrics** (hand / sim):



ID	Completion	Turnaround	Waiting	Response
P1	15 / 15	15 / 15	7 / 7	0 / 0
P2	8 / 8	7 / 7	3 / 3	0 / 0
P3	4 / 4	2 / 2	0 / 0	0 / 0
P4	5 / 5	1 / 1	0 / 0	0 / 0
Average	–	6.25 / 6.25	2.50 / 2.50	0.00 / 0.00

Comparison: the hand-calculated timeline and every metric match the simulator output exactly; no discrepancy.

Example 6: Workload B, SRTF (preemptive)

Input

Process	Arrival	CPU burst
P1	2	3
P2	2	1
P3	8	2
P4	9	1

Hand-calculated timeline: idle 0–2, P2 2–3, P1 3–6, idle 6–8, P3 8–10, P4 10–11.

Same schedule as SJF. At t=2 P2 (1) is shorter than P1 (3). P1 then runs 3–6 with nothing arriving. At t=9, P3 has remaining time 1 and P4 arrives with burst 1: the remaining times are equal, so by the tie rule the running process (P3) is not preempted. P4 runs 10–11.

Simulator output (Gantt chart) and **metrics** (hand / sim):



ID	Completion	Turnaround	Waiting	Response
P1	6 / 6	4 / 4	1 / 1	1 / 1
P2	3 / 3	1 / 1	0 / 0	0 / 0
P3	10 / 10	2 / 2	0 / 0	0 / 0
P4	11 / 11	2 / 2	1 / 1	1 / 1
Average	–	2.25 / 2.25	0.50 / 0.50	0.50 / 0.50

Comparison: the hand-calculated timeline and every metric match the simulator output exactly; no discrepancy.

Example 7: Workload A, Round Robin (q = 2)

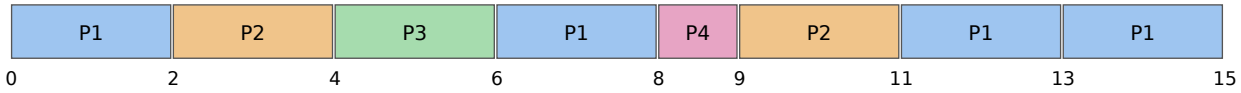
Input

Process	Arrival	CPU burst
P1	0	8
P2	1	4
P3	2	2
P4	4	1

Hand-calculated timeline: P1 0–2, P2 2–4, P3 4–6, P1 6–8, P4 8–9, P2 9–11, P1 11–13, P1 13–15.

Queue trace. t=0: [P1]. P1 runs 0–2 (rem 6). At t=2 P3 arrives at the boundary and joins before P1, so the queue is [P2, P3, P1]. P2 runs 2–4 (rem 2); at t=4 P4 arrives and joins before P2: [P3, P1, P4, P2]. P3 runs 4–6 and finishes. P1 6–8 (rem 4), P4 8–9 finishes, P2 9–11 finishes. P1 is alone: 11–13 and 13–15 (two consecutive quantum slices).

Simulator output (Gantt chart) and **metrics** (hand / sim):



ID	Completion	Turnaround	Waiting	Response
P1	15 / 15	15 / 15	7 / 7	0 / 0
P2	11 / 11	10 / 10	6 / 6	1 / 1
P3	6 / 6	4 / 4	2 / 2	2 / 2
P4	9 / 9	5 / 5	4 / 4	4 / 4
Average	–	8.50 / 8.50	4.75 / 4.75	1.75 / 1.75

Comparison: the hand-calculated timeline and every metric match the simulator output exactly; no discrepancy.

Example 8: Workload B, Round Robin (q = 2)

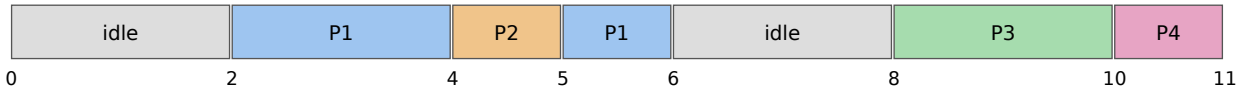
Input

Process	Arrival	CPU burst
P1	2	3
P2	2	1
P3	8	2
P4	9	1

Hand-calculated timeline: idle 0–2, P1 2–4, P2 4–5, P1 5–6, idle 6–8, P3 8–10, P4 10–11.

Idle 0–2. At t=2 the queue is [P1, P2]. P1 runs 2–4 (rem 1) and goes to the tail: [P2, P1]. P2 runs 4–5 and finishes, P1 runs 5–6 and finishes. Idle 6–8. P3 (burst 2) fits in one quantum: 8–10; P4 arrived at 9 and runs 10–11.

Simulator output (Gantt chart) and **metrics** (hand / sim):



ID	Completion	Turnaround	Waiting	Response
P1	6 / 6	4 / 4	1 / 1	0 / 0
P2	5 / 5	3 / 3	2 / 2	2 / 2
P3	10 / 10	2 / 2	0 / 0	0 / 0
P4	11 / 11	2 / 2	1 / 1	1 / 1
Average	–	2.75 / 2.75	1.00 / 1.00	0.75 / 0.75

Comparison: the hand-calculated timeline and every metric match the simulator output exactly; no discrepancy.

3. Additional student-designed test cases

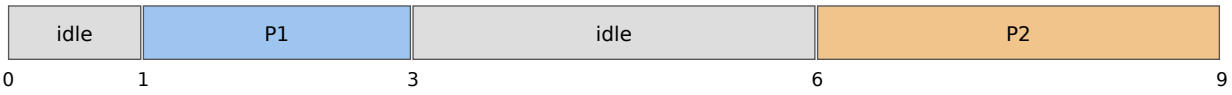
Each test states the behaviour it checks and the expected result, worked out by hand before running the simulator. The expected averages are listed as hand / sim.

Test C: CPU idle time

Input (ID, arrival, burst): P1 (1, 2), P2 (6, 3).

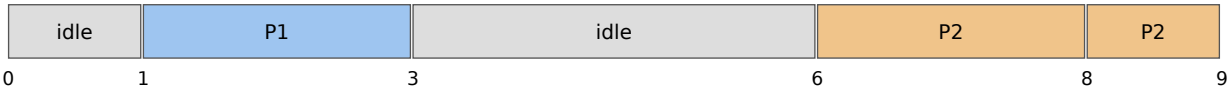
Checks that the CPU is idle before the first arrival (0–1) and between processes (3–6), and that idle intervals appear in the chart. Expected under every algorithm.

FCFS — expected: idle 0–1, P1 1–3, idle 3–6, P2 6–9. Expected averages (turnaround, waiting, response): 2.50, 0.00, 0.00.



Simulator averages: 2.50, 0.00, 0.00. Match.

RR (q = 2) — expected: idle 0–1, P1 1–3, idle 3–6, P2 6–8, P2 8–9. Expected averages (turnaround, waiting, response): 2.50, 0.00, 0.00.



Simulator averages: 2.50, 0.00, 0.00. Match.

Test D: simultaneous arrivals and a tie

Input (ID, arrival, burst): Input order P3 (0, 3), P1 (0, 3), P2 (0, 1).

All three arrive at time 0. P3 and P1 have equal bursts and equal arrival times, so the tie is broken by position in the input list (P3 before P1), not by ID. This catches an implementation that sorts by ID. SJF must also pick the shortest job P2 first.

FCFS — expected: P3 0–3, P1 3–6, P2 6–7. Expected averages (turnaround, waiting, response): 5.33, 3.00, 3.00.



Simulator averages: 5.33, 3.00, 3.00. Match.

SJF — expected: P2 0–1, P3 1–4, P1 4–7. Expected averages (turnaround, waiting, response): 4.00, 1.67, 1.67.



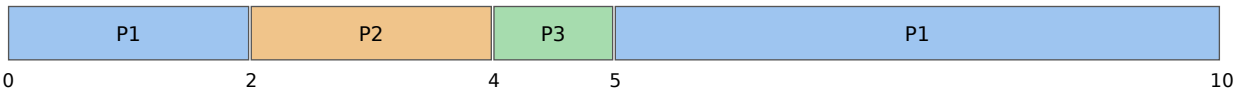
Simulator averages: 4.00, 1.67, 1.67. Match.

Test E: SRTF preempts a running process

Input (ID, arrival, burst): P1 (0, 7), P2 (2, 2), P3 (3, 1).

At t=2, P2 (burst 2) is strictly shorter than P1's remaining 5, so P1 is preempted. At t=3, P2 has remaining 1 and P3 arrives with 1: equal, so P2 keeps running. SJF is shown for contrast (no preemption; P3 is shorter than P2 when P1 finishes).

SRTF — expected: P1 0–2, P2 2–4, P3 4–5, P1 5–10. Expected averages (turnaround, waiting, response): 4.67, 1.33, 0.33.



Simulator averages: 4.67, 1.33, 0.33. Match.

SJF — expected: P1 0–7, P3 7–8, P2 8–10. Expected averages (turnaround, waiting, response): 6.67, 3.33, 3.33.



Simulator averages: 6.67, 3.33, 3.33. Match.

4. Verification of required properties

The simulator performs five checks after each run. They were applied to all 30 runs (five workloads × FCFS, SJF, SRTF and RR with q = 1, 2, 4), using the same code path as the web page (the Node script *tests.js* in the submission reproduces this).

Property	Result
Every process gets exactly its burst	Passed in all runs
No process runs before its arrival	Passed in all runs
No overlap, no gaps in the timeline	Passed in all runs
Waiting and response times are nonnegative	Passed in all runs
CPU never idle while a process is ready	Passed in all runs

5. Analysis

Question 1. For Workload A, which algorithm gives the lowest average waiting time? Which gives the lowest average response time? Explain the scheduling decisions behind the results.

SRTF gives both the lowest average waiting time (2.50, versus 4.75 for RR q=2, 5.25 for SJF and 6.75 for FCFS) and the lowest average response time (0.00). SRTF always runs the process with the least remaining time, so the short jobs P2, P3 and P4 pass the long P1, which absorbs the delay (it waits 7 while they wait 3, 0 and 0). Every process starts the moment it arrives, so all response times are 0. SJF cannot interrupt P1, so the three short jobs wait for it to finish; FCFS does the same and also runs the longer jobs first.

Question 2. How do the round-robin results change when the quantum is 1, 2 and 4? Explain why a smaller quantum does not necessarily improve every metric.

Quantum	Avg turnaround	Avg waiting	Avg response
q = 1	8.50	4.75	0.75
q = 2	8.50	4.75	1.75
q = 4	9.25	5.50	3.75

Response time improves as the quantum shrinks (3.75, 1.75, 0.75) because processes reach the CPU sooner. Waiting and turnaround do not: q=1 and q=2 give identical averages (4.75 and 8.50) and q=4 is worse (5.50 and 9.25). The total work is fixed, so shorter slices mostly move delay between processes: short jobs finish sooner, but the long P1 is pushed back by about as much. A real system also pays switching overhead, which this simulator ignores, so a very small quantum can make every metric worse.

Question 3. Under what conditions do SJF and SRTF produce the same schedule? Give an example.

They are the same when no arriving process ever has a burst strictly shorter than the running process's remaining time, so SRTF never preempts (for example, when all processes arrive together). Example: Workload B. Both give idle 0–2, P2 2–3, P1 3–6, idle 6–8, P3 8–10, P4 10–11; at t=9, P4 has burst 1 and P3 has 1 left, which is not strictly shorter. In Workload A they differ (average waiting 5.25 vs 2.50).

Question 4. Why must a real operating system estimate future CPU bursts, while this simulator can use exact values?

A real OS cannot know in advance how long a process will use the CPU before it blocks, because that depends on input data and program behaviour. It must predict the next burst from past ones (for example with an exponentially weighted average), and a wrong prediction can pick the wrong process. The simulator is given exact bursts, so it shows the ideal behaviour of the policies.

Question 5. Can starvation occur under SJF or SRTF if new short jobs keep arriving? Explain how this differs from the finite workloads used here.

Yes. A long job runs only when no shorter job is ready, so a continuous stream of short arrivals can postpone it indefinitely; under SRTF it can also be preempted repeatedly. In the finite workloads here the arrivals eventually stop, so every process runs after a finite wait: P1 in Workload A is delayed but finishes at 15 under SRTF. Aging avoids starvation.

Question 6. Describe one implementation mistake you encountered and how a test exposed it.

While designing Test E I first wrote the expected SJF order as P1, P2, P3. The simulator gave P1 0–7, P3 7–8, P2 8–10, and re-tracing showed it was right, because P3 (burst 1) is shorter than P2 (burst 2). The error was in my hand expectation, and the comparison against the simulator exposed it. A related bug the tests guard against is in Round Robin: enqueueing the preempted process before a process arriving at the same boundary would put P2 before P4 in Workload A (q=2) and break the match in Example 7.

6. AI-use statement

Tools and purpose. I used Claude (Anthropic) to help write the simulator code and the Node test script, to work out expected results for the workloads, and to draft this report.

How I checked the output. I traced each algorithm by hand for Workloads A and B before relying on the simulator and compared the two for all eight examples; I ran the automated script over five workloads and six algorithm settings with the five property checks (exact burst, no run before arrival, no overlap, nonnegative waiting and response, no idle while ready); and I checked the tie-breaking and round-robin boundary rules against the assignment text.

A suggestion I corrected. The generated expectation for Test E under SJF listed P2 before P3. I rejected it after the comparison showed P3 (shorter burst) must run first, and I corrected the expected result as described in analysis question 6. I remain responsible for the correctness of everything submitted and can trace the simulator’s decisions on an unfamiliar workload.